

EE345 Mod4-Lec2: Error Analysis and Model Selection

[VMLS, 14]

Prediction Errors

For a given data point (x, y) with predicted outcome $\hat{y} = f(x)$, there are four possibilities:

1. True Positive (TP): $y = +1$ and $\hat{y} = +1$ [correct]
2. True Negative (TN): $y = -1$ and $\hat{y} = -1$ [correct]
3. False Positive (FP): $y = -1$ and $\hat{y} = +1$ [incorrect , type I error]
4. False Negative (FN): $y = +1$ and $\hat{y} = -1$ [incorrect, type II error]

Confusion Matrix

	prediction		
$y \backslash \hat{y}$	+1	-1	total
+1	N_{tp}	N_{fn}	N_p
-1	N_{fp}	N_{tn}	N_n
all	$N_{tp} + N_{fp}$	$N_{fn} + N_{tn}$	N

- *Good classifier:*
 - ▶ small (near zero) error rate and false positive rate,
 - ▶ high (near one) true positive rate, true negative rate, and precision.
 - ▶ well calibrated predictions.

		Predicted	
		+1	-1
Actual	+1	TP true positive	(type I error) FP false positive
	-1	(type II error) FN false negative	TN true negative

Counting Errors

Consider data set $(x_1, \dots, x_N)$, $(y_1, \dots, y_N)$ and model f with $N = N_p + N_n$ outcomes.

- **Error rate:**

$$\frac{N_{\text{fp}} + N_{\text{fn}}}{N}$$

- **False positive rate (false alarm):**

$$\frac{N_{\text{fp}}}{N_n}$$

- **True negative rate (specificity):**

$$1 - \frac{N_{\text{fp}}}{N_n} = \frac{N_{\text{tn}}}{N_n}$$

		Predicted	
		+1	-1
Actual	+1	N_{tp} # true positives $y = +1$ and $\hat{y} = +1$	N_{fp} # false positives $y = -1$ and $\hat{y} = +1$
	-1	N_{fn} # false negatives $y = +1$ and $\hat{y} = -1$	N_{tn} # true negatives $y = -1$ and $\hat{y} = -1$

More Error Rates & Metrics

- $N_p = N_{tp} + N_{fn}$: # of outcomes with a true positive label $y = +1$
- $N_n = N_{fp} + N_{tn}$: # of outcomes with a true negative label $y = -1$
- **True positive rate** (sensitivity/recall):

$$\frac{N_{tp}}{N_p}$$

- **Precision:**

$$\frac{N_{tp}}{N_{tp} + N_{fp}} \quad \text{i.e., fraction of true predictions that are correct.}$$

- **F1 score:** harmonic mean of precision and recall

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 N_{tp}}{2 N_{tp} + N_{fp} + N_{fn}}.$$

F1 Score: Balancing Precision and Recall

Why Do We Need It?

- Precision and recall often move in *opposite* directions.
- F1 gives a single number that increases only when *both* are high.
- Uses a harmonic mean to penalize imbalance: precision $\gg$ recall $\implies$ F1 is small.

When to Use F1?

- Positive class is rare (imbalanced datasets).
- Care about detecting positives, but also want predictions to be reliable.
- Do *not* care about TNs (true negatives) as much.
- Often used in:
 - ▶ information retrieval,
 - ▶ medical diagnosis,
 - ▶ fraud/anomaly detection.

Precision, Recall, False Alarm, and ROC

Key Quantities (for positive class $y = +1$)

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall (TPR)} = \frac{TP}{TP + FN}, \quad \text{False Alarm Rate (FPR)} = \frac{FP}{FP + TN}.$$

When to Care About Which?

- **Precision** (How trustworthy are positive predictions?)
 - ▶ Important when false positives are costly; *Example*: flagging emails as spam, fraud alerts.
- **Recall / TPR** (How many real positives did we catch?)
 - ▶ Important when missing a positive is costly; *Example*: disease screening, safety-critical detection.
- **False Alarm Rate / FPR** (How often do we cry wolf?)
 - ▶ Important when too many negatives are incorrectly flagged; *Example*: security systems, anomaly detection.

Iris Data Set Example: Confusion Matrix

- Precision:

$$\frac{N_{tp}}{N_{tp} + N_{fp}}$$

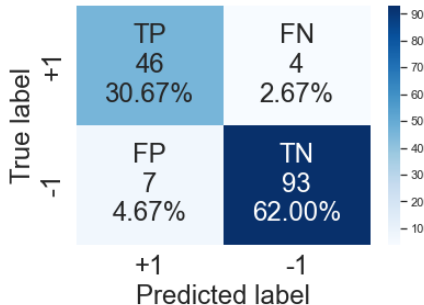
- Accuracy:

$$\frac{N_{tp} + N_{tn}}{N}$$

- F1-score is the harmonic mean of precision (P) and recall (R):

$$\frac{2PR}{P+R}$$

- ▶ recall: $N_{tp}/(N_{tp} + N_{fn})$
- ▶ want it near 1



Accuracy=0.927; Precision=0.959
Recall=0.930; F1 Score=0.944

Two Curves: ROC and Precision–Recall

- ROC curve shows **Recall (TPR)** $N_{tp}/(N_{tp} + N_{fn})$ vs **FPR** $N_{fp}/(N_{fp} + N_{tn})$ across thresholds.
- Precision–Recall curve shows how the reliability of your positive predictions changes as recall increases (ability to catch positives).

Receiver Operating Characteristic (ROC) Curves

Many classifiers output a **score** or **probability**:

$$s_i = p(y_i = 1 \mid x_i).$$

To turn scores into predictions, choose a **threshold** t and predict:

$$\hat{y}_i(t) = \mathbf{1}\{s_i \geq t\}.$$

Receiver Operating Characteristic (ROC) curve:

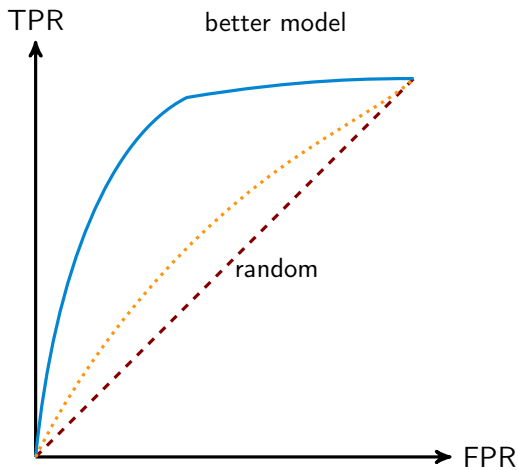
$$\text{ROC} = \{(\text{FPR}(t), \text{TPR}(t)) : t \in (-\infty, +\infty)\}.$$

It plots the tradeoff between false positives and true positives across all possible thresholds.

ROC curve shows how TPR changes as we vary the threshold t . Useful when:

- classes are imbalanced,
- the cost of FP vs FN differs,
- we want threshold-independent comparison.

Example ROC Curves

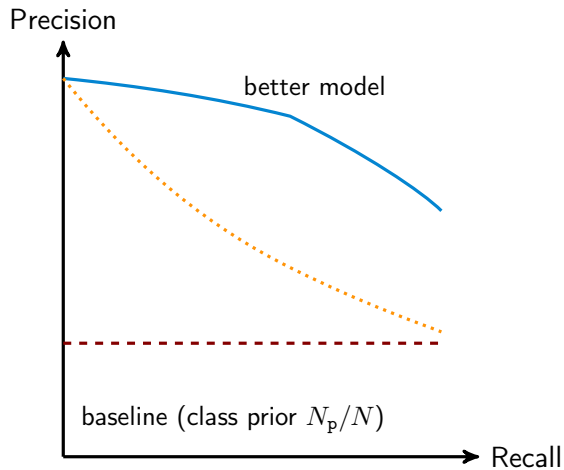


For each threshold t :

$$\text{TPR}(t) = \frac{N_{\text{tp}}(t)}{N_{\text{tp}}(t) + N_{\text{fn}}(t)}$$

$$\text{FPR}(t) = \frac{N_{\text{fp}}(t)}{N_{\text{fp}}(t) + N_{\text{tn}}(t)}$$

Example Precision–Recall Curves



$$\text{Precision} = \frac{N_{\text{tp}}(t)}{N_{\text{tp}}(t) + N_{\text{fp}}(t)},$$

$$\text{Recall (TPR)} = \frac{N_{\text{tp}}(t)}{N_{\text{tp}}(t) + N_{\text{fn}}(t)},$$

AUC: A Threshold-Independent Score

AUC = area under the ROC curve.

Interpretation:

- $\text{AUC} = 1$: perfect classifier.
- $\text{AUC} = 0.5$: random guessing.
- $\text{AUC} = \Pr(\text{random positive} > \text{random negative in score})$ (random guessing).

Why useful?

- Does not depend on any particular threshold.
- Robust to class imbalance.
- Measures how well scores *rank* positives above negatives.

In practice, ROC curves + AUC are standard for evaluating classifiers with probabilistic outputs (e.g. logistic regression).

What Is Probability Calibration?

A classifier outputs predicted probabilities $\hat{p}(x)$.

Calibration means:

$$\Pr(y = 1 \mid \hat{p}(X) \approx p) \approx p \quad \text{for all } p \in [0, 1].$$

Examples:

- Among all predictions near $\hat{p} = 0.2$, about 20% should actually be positive.
- Among all predictions near $\hat{p} = 0.8$, about 80% should be positive.

Well-calibrated models:

- logistic regression (usually),
- naive Bayes (poor),
- random forests (medium),
- deep nets (overconfident, often poorly calibrated).

Calibration is **separate** from accuracy.

How to Compute a Calibration Curve (Reliability Diagram)

Given predicted probabilities p_i and true labels y_i :

1. Split $[0, 1]$ into bins, e.g.

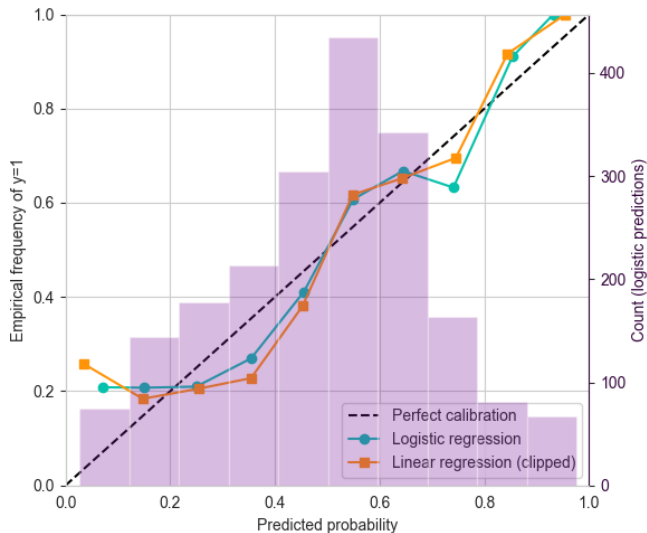
$$[0, 0.1), [0.1, 0.2), \dots, [0.9, 1.0].$$

2. For each bin B :

- ▶ $\hat{p}_B$ = average predicted probability in the bin,
- ▶ $\hat{y}_B$ = empirical fraction of positives in the bin.

3. Plot points $(\hat{p}_B, \hat{y}_B)$.

Calibration Curve Interpretation



Interpretation:

- **Perfect calibration:** points lie on the diagonal $y = x$.
- Above diagonal $\implies$ underconfident.
- Below diagonal $\implies$ overconfident.

Why Does Calibration Matter?

Decision-making depends on probabilities.

Examples:

- Medical diagnosis: $\hat{p} = 0.9$ should *really* mean 90% risk.
- Fraud detection: thresholding depends on calibrated scores.
- Multi-class classification: softmax outputs are often overconfident.

A classifier can have:

- high accuracy but poor calibration,
- good calibration but mediocre accuracy.

ROC curves measure ranking quality. Calibration measures probability quality.

Both are important, but different.

ROC Curves vs Calibration: When to Use Which?

Question	Use ROC / AUC	Use Calibration Curve
How well does the model <i>rank</i> positives above negatives?	Y: ROC/AUC measure ranking quality	
Predicted probabilities <i>meaningful</i> ?		Y: calibration measures probability accuracy
Class imbalance (rare positives)?	ROC/AUC are robust (insensitive) to imbalance	Calibration still works, but bins may need merging
Threshold selection (varying decision threshold)?	ROC ideal: shows tradeoff across all thresholds	Not designed for threshold choice

ROC Curves vs Calibration: When to Use Which?

Question	Use ROC / AUC	Use Calibration Curve
Decision-making under asymmetric costs (e.g. FP vs FN)?	ROC helps visualize tradeoffs	Calibration helps if decisions use probability estimates
Model comparison when only ranking matters (e.g. search, ranking tasks)?	ROC/AUC preferred	
Model comparison when downstream tasks use predicted probabilities (e.g. risk prediction)?		Calibration is essential
Neural networks (often overconfident)?	AUC may still look good	Calibration will reveal overconfidence

Summary: ROC curves evaluate *ranking*. Calibration curves evaluate *probability quality*. Both measure different, and complementary, aspects of model performance.

Cross Validation for Model Selection

Why Cross Validation?

Given data $\{(x_i, y_i)\}_{i=1}^n$, we want to estimate how well a model generalizes.

Problem: If we train and evaluate on the same data:

- training error is overly optimistic,
- overfitting looks good,
- we cannot compare models fairly.

Solution: Split the data into

- a set used for **training**, and
- a set used only for **validation**.

But a single split may be:

- unlucky,
- unstable (high variance),
- too small for training or validation.

Cross validation averages over many splits to stabilize the estimate.

Definition: k -Fold Cross Validation

k -fold cross validation works as follows:

1. Partition the dataset into k equal-sized folds:

$$D := \{(x_i, y_i)\}_{i=1}^n = F_1 \cup F_2 \cup \dots \cup F_k.$$

2. For each $j = 1, \dots, k$:

- ▶ Train on the other $k - 1$ folds:

Train on $D \setminus F_j$,

- ▶ Validate on fold F_j .

- ▶ Record the validation error E_j .

3. Compute metrics: e.g.,

$$\text{avg_error} = \frac{1}{k} \sum_{j=1}^k E_j$$

This gives a low-variance estimate of generalization performance.

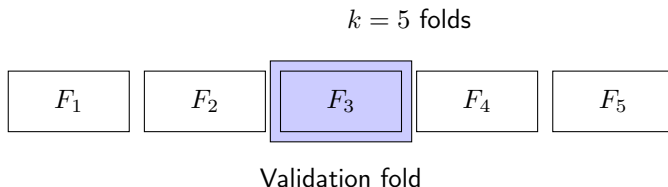
Other CV Metrics

$$\text{avg_error} = \frac{1}{k} \sum_{j=1}^k E_j \quad \text{and} \quad \text{std_error} = \sqrt{\frac{1}{k-1} \sum_{j=1}^k (E_j - \text{avg_error})^2}$$

$$\text{max_error} = \max_{j=1,\dots,k} E_j, \quad \text{min_error} = \min_{j=1,\dots,k} E_j,$$

$$\text{median_error} = \text{median}_{j=1,\dots,k} E_j.$$

Visualization of k -Fold Cross Validation



$$\text{Training} = F_1, F_2, F_4, F_5$$

In each round, we rotate the validation fold and retrain the model. After k rounds, we average the k validation errors.

Cross Validation for Model Selection

Suppose we want to choose amongst the following hyperparameters:

- regularization strength λ ,
- number of features p (or d),
- kernel bandwidth κ ,
- model class $\mathcal{C}$ (e.g. linear vs. polynomial vs. neural network).

For each candidate model $\mathcal{M}(\lambda, p, \kappa, \mathcal{C})$:

- run k -fold CV,
- compute its CV error `error_metric` ($\mathcal{M}(\lambda, p, \kappa, \mathcal{C})$).

Aside: Kernel Bandwidth (i.e. parameterization of a kernel)

In kernel methods (kernel regression, density estimation, RBF kernels), we use a kernel $K_\kappa(x, x_i)$ to measure the “similarity” between points. The parameter $\kappa = \text{bandwidth}$ controls how wide or narrow the kernel is.

Example (Gaussian kernel):

$$K_\kappa(x, x_i) = \exp\left(-\frac{\|x - x_i\|^2}{2\kappa^2}\right).$$

Interpretation of bandwidth:

- **Small** κ : kernels are narrow $\implies$ very local, high flexibility, can overfit
- **Large** κ : kernels are wide $\implies$ heavy smoothing, risk of underfitting

Cross Validation for Model Selection

Choose the model with the lowest CV error:

$$\mathcal{M}^* = \arg \min_{\mathcal{M}(\lambda, p, \kappa, \mathcal{C})} \text{error_metric}(\mathcal{M}(\lambda, p, \kappa, \mathcal{C})).$$

The final model is then:

- retrained on the *full* dataset using $\mathcal{M}^*$,
- ready for test-time deployment.

Why Not Just One Train/Validation Split?

A single validation split has problems:

- **High variance:** the estimate depends on one partition.
- **Wasted data:** validation set not used for training.
- **Unfair to models:** one split may favor some models.
- **Unstable hyperparameters.**

Cross validation:

- uses *all* data for training and validation (on rotation),
- yields a more reliable estimate of generalization,
- reduces randomness from dataset splits,
- improves hyperparameter selection.

Choosing k and Practical Considerations

Common choices:

- $k = 5$ or 10 : good bias/variance tradeoff.
- $k = n$: **Leave-One-Out CV** (LOOCV) — high variance, costly.
- $k = 2$: simple split (train/test).

Practical notes:

- Shuffle data before splitting.
- Stratify folds for classification:

class proportions preserved in each fold.

- Use the **same folds** for all models being compared.
- After selecting hyperparameters, retrain on all data.

Cross validation is the gold standard for model selection in classical ML.

Extra Slides: Maximum Likelihood Estimation

From Probabilistic Modeling to Maximum Likelihood

In logistic regression, we model the conditional distribution:

$$p(y_i = 1 \mid x_i; \theta) = \sigma(\theta^\top x_i), \quad p(y_i = 0 \mid x_i; \theta) = 1 - \sigma(\theta^\top x_i),$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$.

Assumption: Each label y_i is generated independently from a **Bernoulli** distribution:

$$y_i \sim \text{Bernoulli}(\sigma(\theta^\top x_i)).$$

Therefore, the likelihood is:

$$L(\theta) = \prod_{i=1}^n \left[\sigma(\theta^\top x_i) \right]^{y_i} \left[1 - \sigma(\theta^\top x_i) \right]^{1-y_i}.$$

This is the probability of observing the labels $\{y_i\}$ under the model.

What Is a Likelihood?

Suppose we have:

- data (observations) D ,
- a parametric model $p(D \mid \theta)$.

Definition (Likelihood). The *likelihood function* is

$$L(\theta) := p(D \mid \theta),$$

viewed as a function of the parameter θ with the data D fixed.

- Probability: a function of the **observed outcomes**.
- Likelihood: the **same quantity**, but viewed as a function of **parameters**.

Goal in Maximum Likelihood: Find parameter θ that makes the observed data most probable:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} L(\theta).$$

Likelihood Example: A Biased Coin

Suppose we flip a coin n times and see outcomes $y_1, \dots, y_n \in \{0, 1\}$.

Model:

$$y_i \sim \text{Bernoulli}(p), \quad p = \text{Pr}(\text{Heads}).$$

Probability of the whole dataset:

$$p(y_1, \dots, y_n \mid p) = \prod_{i=1}^n p^{y_i} (1 - p)^{1-y_i}.$$

As a function of p , this is the **likelihood**:

$$L(p) = p^{\sum_i y_i} (1 - p)^{n - \sum_i y_i}.$$

Interpretation: This tells us how plausible different values of p are, *given the data we observed*.

Likelihood in Supervised Learning

In supervised learning, we observe pairs (x_i, y_i) .

To define a likelihood, we must:

1. Choose a model for $p(y \mid x, \theta)$;
2. Assume the training examples are **independent**.

Then the joint probability of all labels given inputs $X = \{x_i\}$ and parameters θ is:

$$p(y_1, \dots, y_n \mid x_1, \dots, x_n, \theta) = \prod_{i=1}^n p(y_i \mid x_i, \theta).$$

This joint probability, treated as a function of θ , is the **likelihood**:

$$L(\theta) = \prod_{i=1}^n p(y_i \mid x_i, \theta).$$

Maximum likelihood picks θ making the observed labels most probable.

Likelihood for Logistic Regression

Model the conditional distribution:

$$p(y_i = 1 \mid x_i, \theta) = \sigma(\theta^\top x_i), \quad p(y_i = 0 \mid x_i, \theta) = 1 - \sigma(\theta^\top x_i).$$

Assumption: The labels $y_1, \dots, y_n$ are conditionally independent given the inputs.

Joint probability of the dataset:

$$p(y_1, \dots, y_n \mid x_1, \dots, x_n, \theta) = \prod_{i=1}^n \left[\sigma(\theta^\top x_i) \right]^{y_i} \left[1 - \sigma(\theta^\top x_i) \right]^{1-y_i}.$$

This is the likelihood:

$$L(\theta) = \prod_{i=1}^n \left[\sigma(\theta^\top x_i) \right]^{y_i} \left[1 - \sigma(\theta^\top x_i) \right]^{1-y_i}.$$

Maximizing this is equivalent to minimizing the logistic loss.

Temporary page!

$\text{\LaTeX}$ was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because now knows how many pages to expect for this document.